

1. a) Write a C program to implement recursive binary search.

b) Write a C program to sort numbers using selection sort algorithm.

Program a :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int BinarySearch(int list[],int lo,int hi,int key);
```

```
int compare(int m,int k);
```

```
int main()
```

```
{
```

```
    int arr[20],n;
```

```
    int search,location,low,high;
```

```
    printf("Enter the number of elements : ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter elements : ");
```

```
    for(int i=0;i<n;i++)
```

```
        scanf("%d",&arr[i]);
```

```
    printf("Enter the search element : ");
```

```
    scanf("%d",&search);
```

```
    low=0;high=n-1;
```

```
    location=BinarySearch(arr,low,high,search);
```

```
    if(location>=0)
```

```
        printf("Searched element is at location %d",location);
```

```
    else
```

```
        printf("No element found");
```

```
    return 0;
```

```
}
```

```
int BinarySearch(int list[],int lo,int hi,int key)
```

```
{
```

```

int mid;
if(lo>hi)
{
    printf("Key not found\n");
    return -1;
}
else
{
    mid=(int)(lo+hi)/2;
    switch(compare(list[mid],key))
    {
        case 0 :
            printf("Key Found\n");
            return mid;
        case 1 : //list of mid > key
            return BinarySearch(list,lo,mid-1,key);
        case -1 ://list of mid < key
            return BinarySearch(list,mid+1,hi,key);
    }
}
return -1;
}

int compare(int m,int k)
{
    if(m<k)
        return -1;
    else if(m>k)
        return 1;
    else
        return 0;
}

```

Output :

Enter the number of elements : 5

Enter elements : 20 40 60 80 100

Enter the search element : 40

Key Found

Searched element is at location 1

Program b :

```
#include <iostream>

using namespace std;

void SwapIt(int *a,int *b);

void ssort(int arr[],int ele);

int main()
{
    int arr[10],ele;           //increase array size if required
    cout<<"ENTER THE NO OF ELEMENTS(2-10) : ";
    cin>>ele;
    cout<<"ENTER THE ELEMENTS : ";
    for(int i=0;i<ele;i++)
        cin>>arr[i]; //read elements

    ssort(arr,ele);

    cout<<endl<<"SORTED ELEMENTS ARE : ";
    for(int i=0;i<ele;i++)
    {
        cout<<arr[i]<<"\t";
    }
    return 0;
}

void SwapIt(int *a,int *b)
{

```

```

        int temp=*a;
        *a=*b;
        *b=temp;
    }
    void ssort(int arr[],int ele)
    {
        int mini;
        for(int i=0;i<ele-1;i++)
        {
            mini=i;
            for(int j=i+1;j<ele;j++)
            {
                if(arr[mini]>arr[j])
                {
                    mini=j;
                }
            }
            SwapIt(&arr[i],&arr[mini]);
        }
    }
}

```

Output :

ENTER THE NO OF ELEMENTS(2-10) : 5

ENTER THE ELEMENTS : 80 40 20 60 100

SORTED ELEMENTS ARE : 20 40 60 80 100

2. Write a C program to perform addition of two polynomials using single array:

a) Store two polynomials in single array (10 marks)

b) Perform Addition of two polynomials (20 marks)

c) Display the result (5 marks)

Program :

```
#include <stdio.h>
#include <stdlib.h>
int MAXTERM=100;
int avail=0;
typedef struct
{
    int exp;
    float coeff;
}polynomials;
polynomials terms[100];

void attach(float coefficient , int exponent);
int compare(int aexp,int bexp);
void display(int* startd,int* finishd);
void padd(int starta,int finisha,int startb,int finishb,int* startd,int* finishd);

void attach(float coefficient , int exponent)
{
    if(avail>=MAXTERM)
    {
        printf("To many arguements ");
        exit(1);
    }
    terms[avail].coeff=coefficient;
    terms[avail++].exp=exponent;
}
```

```

int compare(int aexp,int bexp)
{
    if(aexp<bexp)
        return -1;
    else if(aexp>bexp)
        return 1;
    else
        return 0;
}

void display(int* startd,int* finishd)
{
    printf("Addition of two polynomial is \n");
    for(int i=*startd;i<=*finishd;i++)
    {
        printf("%fX^%d",terms[i].coeff,terms[i].exp);
        if(i!=*finishd)
            printf(" + ");
    }
}

void padd(int starta,int finisha,int startb,int finishb,int* startd,int* finishd)
{
    float coefficient;
    *startd=avail; //to mark start index
    while(starta<=finisha&&startb<=finishb)
    {
        switch(compare(terms[starta].exp,terms[startb].exp))
        {
            case -1 : //if a-exponent < b-exponent
                attach(terms[startb].coeff,terms[startb].exp);
                startb++;
                break;
            case 0 ://if a-exponent == b-exponent

```

```

        coefficient=terms[starta].coeff+terms[startb].coeff;
        if(coefficient)
            attach(coefficient,terms[starta].exp);
        starta++;startb++;
        break;
    case 1 ://if a-exponent > b-exponent
        attach(terms[starta].coeff,terms[starta].exp);
        starta++;
    }
}
for(;starta<=finisha;starta++) //to attach all remaining elements of a , if any
    attach(terms[starta].coeff,terms[starta].exp);
for(;startb<=finishb;startb++) //to attach all remaining elements of b , if any
    attach(terms[startb].coeff,terms[startb].exp);
*finishd=avail-1; //to mark end index
}

```

```

int main()
{
    int nA,nB;
    int mexp;float mcoeff;
    int s,f;
    printf("Enter number of elements in polynomial A : ");
    scanf("%d",&nA);
    printf("Enter number of elements in polynomial B : ");
    scanf("%d",&nB);

    printf("Enter the coefficient and exponent of A \n");
    for(int i=0;i<nA;i++)
    {
        scanf("%f%d",&mcoeff,&mexp);
        attach(mcoeff,mexp);
    }
}

```

```

    }
    printf("Enter the coefficient and exponent of B \n");
    for(int i=0;i<nB;i++)
    {
        scanf("%f%d",&mcoeff,&mexp);
        attach(mcoeff,mexp);
    }

    padd(0,nA-1,nA,nA+nB-1,&s,&f);

    display(&s,&f);

    return 0;
}

```

Output :

Enter number of elements in polynomial A : 3

Enter number of elements in polynomial B : 4

Enter the coefficient and exponent of A

10 2

6 1

3 0

Enter the coefficient and exponent of B

4 3

15 2

4 1

2 0

Addition of two polynomial is

$4.000000X^3 + 25.000000X^2 + 10.000000X^1 + 5.000000X^0$

3. Write and implement a C program to perform transpose of Sparse Matrix using

a) Simple Transpose Method (15 marks)

b) Fast Transpose Method (20 marks)

Program :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct terms
{
    int row;
    int col;
    int val;
}terms;

void read(int r,int c,int list[][c]);
void check(int r,int c,int* k,int list[][c],terms s[]);
void display(int k,terms s[]);
void ntrans(terms a[],terms b[]);
void ftrans(terms a[],terms b[]);

void read(int r,int c,int list[][c])
{
    printf("Enter the values : \n");
    for(int i=0;i<r;i++)
    {
        for(int j=0;j<c;j++)
        {
            scanf("%d",&list[i][j]);
        }
    }
}
```

```

}
void check(int r,int c,int* k,int list[][c],terms s[])
{
    *k=1;
    for(int i=0;i<r;i++)
    {
        for(int j=0;j<c;j++)
        {
            if(list[i][j]!=0)
            {
                s[*k].row=i;
                s[*k].col=j;
                s[*k].val=list[i][j];
                *k=*k+1;
            }
        }
    }
    s[0].row=r; s[0].col=c; s[0].val=*k-1;
}
void display(int k,terms s[])
{
    printf("Sparse matrix is \n");
    printf("\nRow    column    value\n");
    for(int i=0;i<k;i++)
    {
        printf(" %d\t%d\t%d\n",s[i].row,s[i].col,s[i].val);
    }
}
void ntrans(terms a[],terms b[])
{
    int n,i,j,currentb;
    n=a[0].val;

```

```
b[0].row=a[0].col;  
b[0].col=a[0].row;  
b[0].val=a[0].val; //or equal to n also
```

```
if(n>0) //non zero matrix
```

```
{  
    currentb=1;  
    for(i=0;i<a[0].col;i++)  
    {  
        for(j=1;j<=n;j++)  
        {  
            if(a[j].col==i)  
            {  
                b[currentb].row=a[j].col;  
                b[currentb].col=a[j].row;  
                b[currentb].val=a[j].val;  
  
                currentb++;  
            }  
        }  
    }  
}
```

```
}
```

```
void ftrans(terms a[],terms b[])
```

```
{  
    int maxcol=50;  
  
    int row_terms[maxcol],start_pos[maxcol];  
    int i,j;  
    int num_cols=a[0].col,num_terms=a[0].val;
```

```
b[0].row=a[0].col; b[0].col=a[0].row; b[0].val=a[0].val;
```

```

if(num_terms>0)    //non zero terms
{
    for(i=0;i<num_cols;i++)
        row_terms[i]=0;
    for(i=1;i<=num_terms;i++)
        row_terms[a[i].col]++;

    start_pos[0]=1;

    for(i=1;i<num_cols;i++)
        start_pos[i] = start_pos[i-1] + row_terms[i-1];
    for(i=1;i<=num_terms;i++) // now onwards no need to use row_terms array
    {
        j=start_pos[a[i].col]++;

        b[j].row=a[i].col;
        b[j].col=a[i].row;
        b[j].val=a[i].val;
    }
}

int main()
{
    int row,col,matrix[20][20],k,select;
    terms sparseA[50],sparseB[50];
    printf("Enter the number of rows and column : ");
    scanf("%d",&row);
    scanf("%d",&col);

    read(row,col,matrix);
    check(row,col,&k,matrix,sparseA);

```

```

display(k,sparseA);

printf("\nEnter \n1 for normal\n2 for fast \n3 for EXIT\n\n");

while(1)
{
    printf("Enter choice : ");
    scanf("%d",&select);
    switch(select)
    {
        case 1 :
            ntrans(sparseA,sparseB);
            display(k,sparseB);
            break;
        case 2 :
            ftrans(sparseA,sparseB);
            display(k,sparseB);
            break;
        case 3:
            printf("Exiting.....\n");
            exit(0);
        default:
            printf("Select correct number \n");
    }
}

return 0;
}

```

Output:

Enter the number of rows and column : 6 6

Enter the values :

15 0 0 22 0 -15

0 11 3 0 0 0

0 0 0 -6 0 0

0 0 0 0 0 0

91 0 0 0 0 0

0 0 28 0 0 0

Sparse matrix is

Row column value

6 6 8

0 0 15

0 3 22

0 5 -15

1 1 11

1 2 3

2 3 -6

4 0 91

5 2 28

Enter

1 for normal

2 for fast

3 for EXIT

Enter choice : 1

Sparse matrix is

Row column value

6 6 8

0 0 15

0 4 91

1 1 11

2 1 3

2 5 28

3 0 22

3 2 -6

5 0 -15

Enter choice : 2

Sparse matrix is

Row column value

6 6 8

0 0 15

0 4 91

1 1 11

2 1 3

2 5 28

3 0 22

3 2 -6

5 0 -15

Enter choice : 3

Exiting.....

4. Write and implement a C program to perform Pattern Matching using.

a) KMP pattern matching (20 marks)

b) nfind matching (15 marks)

Program:

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int failure[100];
```

```
void fail(char fpat[]);
```

```
int patmatch(char fstring[],char fpat[]);
```

```
int nfind(char fstring[],char fpat[]);
```

```
void fail(char fpat[])
```

```
{
```

```
    int i,j;
```

```
    int n = strlen(fpat);
```

```
    failure[0] = -1;
```

```
    for(j=1;j<n;j++)
```

```
    {
```

```
        i = failure [j-1];
```

```
        while((fpat[j]!=fpat[i+1])&&(i>=0))
```

```
            i = failure[i];
```

```
        if(fpat[j]==fpat[i+1])
```

```
            failure[j] = i+1; //array is initialized
```

```
        else
```

```
            failure[j] = -1; //array is initialized
```

```
    }
```

```
}
```

```
int patmatch(char fstring[],char fpat[])
```

```
{
```

```

int i=0,j=0;
int lens=strlen(fstring);
int lenp=strlen(fpat);

while(i<lens&& j<lenp)
{
    if(fstring[i]==fpat[j])
    {
        i++;j++;
    }
    else if(j==0)
        i++;
    else
        j=failure[j-1]+1;
}
return ((j==lenp) ? (i-lenp) : -1);
}

int nfind(char fstring[],char fpat[])
{
    int i,j,start=0; //start is starting index of matched pattern if happened
    int lasts=strlen(fstring)-1; //string - last index value
    int lastp=strlen(fpat)-1; //pattern - last index value
    int endmatch=lastp;

    for(i=0;endmatch<=lasts;endmatch++,start++)
    {
        if(fstring[endmatch]==fpat[lastp])
            for(j=0,i=start;j<lastp&&fstring[i]==fpat[j];i++,j++);
        if(j==lastp)
            return start;
    }
    return -1;
}

```

```

}
int main()
{
    char string[100],pat[100];
    int i=-1,select;
    printf("Enter the string : ");
    scanf("%s",string);
    printf("Enter the pattern : ");
    scanf("%s",pat);

    printf("Enter \n1 for nfind method\n2 for KMP method\n");
    scanf("%d",&select);

    switch(select)
    {
        case 1 :
            i=nfind(string,pat);
            break;
        case 2 :
            fail(pat);
            i=patmatch(string,pat);
            break;
        default:
            printf("Enter proper number !");
    }

    if(i!=-1)
        printf("Match found at %d index of string ",i);
    else
        printf("No match found !");
    //for(int i=0;i<5;i++) //only for KMP and pattern size 5
        //printf("%d ",failure[i]);

```

```
    return 0;  
}
```

Output 1 (nfind) :

Enter the string : abcabcacab

Enter the pattern : abcac

Enter

1 for nfind method

2 for KMP method

1

Match found at 3 index of string

Output 2 (KMP) :

Enter the string : abcabcacab

Enter the pattern : abcac

Enter

1 for nfind method

2 for KMP method

2

Match found at 3 index of string

5. Write and implement a C program to convert the given Infix expression to Postfix expression using stack (35 Marks)

Program :

```
#include<stdio.h>

#define MAX_STACK_SIZE 50
#define MAX_EXPR_SIZE 50

typedef enum {lparen,rparen,plus,minus,times,divide,mod,eos,operand} precedence;

precedence stack[MAX_STACK_SIZE];
static int isp[] = {0,19,12,12,13,13,13,0};
static int icp[] = {20,19,12,12,13,13,13,0};
char expr[MAX_EXPR_SIZE];

precedence get_token(char* symbol,int* n);
void postfix(void);
void addf(int *t,precedence op);
precedence deletf(int *t);
void printfun(precedence i);

void postfix(void)
{
    char symbol;
    precedence token;
    int n = 0;
    int top = 0;
    stack[0] = eos;
    for(token = get_token(&symbol,&n);token!=eos;token = get_token(&symbol,&n))
    {
        if(token==operand)
            printf("%c",symbol);
    }
}
```

```

else if(token==rparen)
{
    while(stack[top]!=lparen)
        printfun(deletetf(&top));
    token=deletetf(&top); //temporary storing value , and not for any purpose
}
else
{
    while(isp[stack[top]] >= icp[token])
        printfun(deletetf(&top));
    addf(&top,token);
}
}
token=deletetf(&top);
while(token!=eos) //print remaining elements
{
    printfun(token);
    token=deletetf(&top);
}
printf("\n");
}
precedence get_token(char* symbol,int* n)
{
    *symbol = expr[*n];
    (*n)++;
    switch(*symbol)
    {
        case '(' : return lparen;
        case ')' : return rparen;
        case '+' : return plus;
        case '-' : return minus;
    }
}

```

```

        case '*' : return times;
        case '/' : return divide;
        case '%' : return mod;
        case '\0' : return eos;
        default : return operand;
    }
}

void addf(int *t,precedence op)
{
    if((*t)==MAX_STACK_SIZE-1)
    {
        printf("Stack is full");
        return;
    }
    (*t)++; //increment of top
    stack[*t]=op;
    return;
}

precedence deletef(int *t)
{
    if((*t)==-1)
    {
        printf("Stack is empty");
        return eos;
    }
    precedence val = stack[*t];
    (*t)--;
    return val;
}

void printfun(precedence i)
{
    switch(i)

```

```

{
    case lparen : printf("("); break;
    case rparen : printf(")"); break;
    case plus : printf("+"); break;
    case minus : printf("-"); break;
    case times : printf("*"); break;
    case divide : printf("/"); break;
    case mod : printf("%"); break;
    case eos: printf(" "); break;
    default : printf(" ");
}
}
int main()
{
    printf("Enter the expression : ");
    scanf("%s",expr);

    postfix();
    return 0;
}

```

Output:

Enter the expression : a*(b+c)*d
abc+*d*

6. Write and implement a C program to perform Evaluation of Postfix expression using stack. (35 Marks)

Program :

```
#include<stdio.h>

#define MAX_STACK_SIZE 50
#define MAX_EXPR_SIZE 50

typedef enum {lparen,rparen,plus,minus,times,divide,mod,eos,operand} precedence;
precedence get_token(char* symbol,int* n);

int stack[MAX_STACK_SIZE]; //it is used to store operands
char expr[MAX_EXPR_SIZE]; //it is a string

int eval();
precedence get_token(char* symbol,int* n);
void addf(int *t,int op);
int deletf(int *t);

int eval()
{
    precedence token;
    char symbol; //used to store each character token
    int op1,op2;
    int n = 0; //used to read in expr string
    int top = -1;
    token = get_token(&symbol,&n);

    while(token!=eos) //checks if expr string is not empty
    {
        if(token==operand)
            addf(&top,symbol-'0'); //if operand then push to stack
```

```

else
{
    op2=deletf(&top);
    op1=deletf(&top);

    switch(token) //pop two elements and solve
    {
        case plus :
            addf(&top,op1+op2);
            break;
        case minus :
            addf(&top,op1-op2);
            break;
        case times :
            addf(&top,op1*op2);
            break;
        case divide :
            addf(&top,op1/op2);
            break;
        case mod :
            addf(&top,op1%op2);
    }
}

token = get_token(&symbol,&n); //assign next character to token
}

return deletf(&top); //once null character is reached then pop result
}

precedence get_token(char* symbol,int* n)
{
    *symbol = expr[(*n)];
    (*n)++;
    switch(*symbol) //symbol is used to check operand

```

```

{           //but as it is required in eval() it is used as pointer variable
    case '(' : return lparen;
    case ')' : return rparen;
    case '+' : return plus;
    case '-' : return minus;
    case '*' : return times;
    case '/' : return divide;
    case '%' : return mod;
    case '\0' : return eos;
    default : return operand;
}
}
void addf(int *t,int op)
{
    if((*t)==(MAX_STACK_SIZE-1))
    {
        printf("Stack is full ! ");
        return;
    }
    (*t)++; //increment of top
    stack[*t]=op;
    return;
}
int deletf(int *t)
{
    if((*t)==-1)
    {
        printf("Stack is empty");
        return 0;
    }
    int val = stack[*t];
    (*t)--;

```

```
        return val;
    }
int main()
{
    int ans;
    printf("Enter the expression : ");
    scanf("%s",expr);
    ans=eval();
    printf("Answer is %d",ans);

    return 0;
}
```

Output:

Enter the expression : 62/3-42*+

Answer is 8

7. Write and implement a C program to perform the following operations on a circular queue using dynamically allocated arrays.

a) Insert an item into circular queue (15 marks)

b) Delete an item from circular queue (10 marks)

c) Display the circular queue contents (10 marks)

Program :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int capacity=5;
```

```
int front=0,rear=0;
```

```
typedef struct element
```

```
{
```

```
    int key;
```

```
}element;
```

```
element *queue;
```

```
void queueFull();
```

```
void addq(element ele);
```

```
element deleteq();
```

```
void display();
```

```
void copy(element* start,element* end,element* nqueue);
```

```
void queueFull()
```

```
{
```

```
    element *newqueue;
```

```
    newqueue = (element*) malloc(2*capacity*sizeof(element));
```

```
    int start = (front+1)%capacity;
```

```
    if(start<2)
```

```

        copy(queue+start,queue+start+capacity-1,newqueue);
    else
    {
        copy(queue+start,queue+capacity,newqueue);
        copy(queue,queue+rear+1,newqueue+capacity-start);
    }
    front = 2*capacity-1;
    rear = capacity-2;
    capacity*=2;
    free(queue);
    queue = newqueue;
}

void addq(element ele)
{
    rear=(rear+1)%capacity;
    if(front==rear)
    {
        printf("\nQueue is full\n");
        printf("\nDoubling size\n");
        queueFull();
    }
    queue[rear]=ele;
}

element deleteq()
{
    element ele;
    ele.key=0;
    if(front==rear)
    {
        printf("\nQueue is empty\n");
        return ele;
    }
}

```

```

    front=(front+1)%capacity;
    ele=queue[front];
    return ele;
}
void display()
{
    for(int i=(front+1)%capacity;i!=(rear+1)%capacity;i=(i+1)%capacity)
        printf(" %d ",queue[i].key);
    printf("\n"); //format output
}
void copy(element* start,element* end,element* nqueue)
{
    element* i;
    element* j;
    i=nqueue;
    j=start;
    for(;j<end;i++,j++)
    {
        *i = *j;
    }
}
void main()
{
    queue = (element*) malloc(capacity*sizeof(element));
    int select;
    element val;
    printf("Enter the operation : \n1 for add \n2 for delete \n3 for display \n4 for exit\n\n");
    while(1)
    {
        printf("Enter choice : ");
        scanf("%d",&select);

```

```

switch(select)
{
    case 1 :
        printf("Enter the element : ");
        scanf("%d",&val.key);
        addq(val);
        break;
    case 2 :
        val=deleteq();
        printf("Deleted element %d\n",val.key);
        break;
    case 3 :
        display();
        break;
    case 4 :
        free(queue);      // free the queue
        exit(0);
        break;
    default :
        printf("Select the correct number");
}
}
return;
}

```

Output :

Enter the operation :

- 1 for add
- 2 for delete
- 3 for display
- 4 for exit

Enter choice : 1

Enter the element : 20

Enter choice : 1

Enter the element : 40

Enter choice : 1

Enter the element : 60

Enter choice : 1

Enter the element : 80

Enter choice : 3

20 40 60 80

Enter choice : 2

Deleted element 20

Enter choice : 3

40 60 80

Enter choice : 1

Enter the element : 100

Enter choice : 1

Enter the element : 120

Queue is full

Doubling size

Enter choice : 1

Enter the element : 140

Enter choice : 3

40 60 80 120 140

Enter choice : 4

8. Write a C program to implement a following stack operations using linked lists.

a) Push an item in stack (15 marks)

b) Pop an item from stack (10 marks)

c) Display stack contents (10 marks)

Program :

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
typedef struct listnode *list_pointer;
```

```
typedef struct listnode
```

```
{
```

```
    int data;
```

```
    list_pointer link;
```

```
}listnode;
```

```
list_pointer add(list_pointer *top,int item);
```

```
int del(list_pointer *top);
```

```
void freeit(list_pointer *top);
```

```
void disp(list_pointer top);
```

```
list_pointer add(list_pointer *top,int item)
```

```
{
```

```
    list_pointer temp;
```

```
    temp = (list_pointer)malloc(sizeof(listnode));
```

```
    if(!(temp))
```

```
    {
```

```
        printf("Memory full");
```

```
        return 0;
```

```
    }
```

```
    temp->data = item;
```

```
    temp->link = *top;
```

```

    return temp;
}
int del(list_pointer *top)
{
    list_pointer temp = *top;
    int item;
    if(!(temp))
    {
        printf("Stack is empty\n");
        return 0;
    }
    item = (*top)->data;
    (*top) = (*top)->link;
    free(temp);
    return item;
}
void freeit(list_pointer *top)//used to free memory
{
    list_pointer temp;
    temp = *top;
    while(*top)
    {
        (*top) = (*top)->link;
        free(temp);
        temp = *top;
    }
    printf("Successfully deleted\n");
}
void disp(list_pointer top)
{
    list_pointer ptr;
    ptr = top;

```

```

printf("Elements are : ");
if(!ptr)          //empty stack
{
    printf("No Elements!\n");
    return;
}
while(ptr!=NULL)
{
    printf("%d ",ptr->data);
    ptr = ptr->link;
}
printf("\n");
}
int main()
{
    list_pointer top = NULL;
    int select,value;
    printf("Enter :\n 1 for push\n 2 for pop\n 3 for display\n 4 for EXIT\n\n");
    while(1)
    {
        printf("Enter choice : ");
        scanf("%d",&select);

        switch(select)
        {
            case 1:
                printf("Enter the value : ");
                scanf("%d",&value);
                top = add(&top,value);
                break;
            case 2:
                value = del(&top);

```

```

        printf("deleted value is %d\n",value);
        break;
    case 3:
        disp(top);
        break;
    case 4:
        printf("Exiting.....\n");
        freeit(&top);
        exit(0);
        break;
    default:
        printf("Enter proper number\n");
    }
}
return 0;
}

```

Output :

Enter :

1 for push

2 for pop

3 for display

4 for EXIT

Enter choice : 1

Enter the value : 10

Enter choice : 1

Enter the value : 20

Enter choice : 1

Enter the value : 30

Enter choice : 1

Enter the value : 40

Enter choice : 3

Elements are : 40 30 20 10

Enter choice : 2

deleted value is 40

Enter choice : 3

Elements are : 30 20 10

Enter choice : 4

Exiting.....

Successfully deleted

9. Write a C program to implement a following queue operations using linked lists

a) Add an item in queue (15 marks)

b) Delete an item from queue (10 marks)

c) Display queue contents (10 marks)

Program :

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
typedef struct
```

```
{
```

```
    int key;
```

```
}element;
```

```
typedef struct queue *queue_pointer;
```

```
typedef struct queue
```

```
{
```

```
    element item;
```

```
    queue_pointer link;
```

```
}queue;
```

```
int IS_FULL(queue_pointer temp);
```

```
int IS_EMPTY(queue_pointer front);
```

```
void addq(queue_pointer *front,queue_pointer *rear,element item);
```

```
element deleteq(queue_pointer *front);
```

```
void disp(queue_pointer *front);
```

```
void freeit(queue_pointer *top);
```

```
int IS_FULL(queue_pointer temp)
```

```
{
```

```
    return !(temp);
```

```
}
```

```

int IS_EMPTY(queue_pointer front)
{
    return !(front);
}

void addq(queue_pointer *front, queue_pointer *rear, element item)
{
    queue_pointer temp;
    temp = (queue_pointer) malloc(sizeof(queue));
    if(IS_FULL(temp))
    {
        printf("memory full");
        exit(1);
    }
    temp->item = item;
    temp->link = NULL;
    if(*front)
        (*rear)->link = temp;
    else
        *front = temp;
    *rear = temp;
}

element deleteq(queue_pointer *front)
{
    queue_pointer temp = *front;
    element item;
    if(IS_EMPTY(*front))
    {
        printf("Queue is empty");
        exit(1);
    }
    item = temp->item;
    *front = temp->link;
}

```

```

    free(temp);
    return item;
}
void disp(queue_pointer *front)
{
    queue_pointer ptr = *front;
    printf("Elements are : ");
    if(!(*front)) //if no elements
    {
        printf("No Elements!");
    }
    while(ptr!=NULL)
    {
        printf("%d ",(ptr->item).key);
        ptr = ptr->link;
    }
    printf("\n");//format output
}
void freeit(queue_pointer *top)
{
    queue_pointer temp;
    temp = *top; //here both are pointers of struct type
    while(temp!=NULL)
    {
        temp = temp->link;
        free(*top);
        *top = temp;
    }
    printf("Successfully deleted");
}
void main()
{

```

```

int select;
element item;
queue_pointer front = NULL;
queue_pointer rear;

printf("Enter\n1 for enqueue\n2 for dequeue\n3 for display\n4 for exit\n\n");
while(1)
{
    printf("Select the number : ");
    scanf("%d",&select);
    switch(select)
    {
        case 1 :
            printf("Enter the element : ");
            scanf("%d",&item.key);
            addq(&front,&rear,item);
            break;
        case 2:
            item = deleteq(&front);
            printf("Deleted element is %d \n",item.key);
            break;
        case 3:
            disp(&front);
            break;
        case 4:
            printf("Exit successfully\n");
            freeit(&front);
            exit(0);
            break;
        default :
            printf("Enter proper number");
    }
}

```

```
}  
    return;  
}
```

Output :

Enter

1 for enqueue

2 for dequeue

3 for display

4 for exit

Select the number : 1

Enter the element : 20

Select the number : 1

Enter the element : 25

Select the number : 1

Enter the element : 30

Select the number : 1

Enter the element : 35

Select the number : 1

Enter the element : 40

Select the number : 3

Elements are : 20 25 30 35 40

Select the number : 2

Deleted element is 20

Select the number : 3

Elements are : 25 30 35 40

Select the number : 4

Exit successfully

Successfully deleted

10. Write and implement a C program to perform the following using linked lists.

a) Create Linked lists to store two polynomials (20 marks)

b) Add two polynomials (15 marks)

Program :

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
typedef struct poly_node *poly_pointer;
```

```
typedef struct poly_node
```

```
{
```

```
    float coef;
```

```
    int expon;
```

```
    poly_pointer link;
```

```
}poly_node;
```

```
void attach(float coefficient,int exponent,poly_pointer *ptr);
```

```
poly_pointer padd(poly_pointer a,poly_pointer b);
```

```
int compare(int a,int b);
```

```
void createnode(poly_pointer *f);
```

```
void display(poly_pointer ptr);
```

```
void erase(poly_pointer *ptr);
```

```
void attach(float coefficient,int exponent,poly_pointer *ptr) //ptr because pass by reference
```

```
{ //ptr is having address of rear pointer
```

```
    poly_pointer temp;
```

```
    temp = (poly_pointer)malloc(sizeof(poly_node)); //create a memory block
```

```
    if(!temp)
```

```
    {
```

```
        fprintf(stderr,"The memory is full\n");
```

```
        exit(1);
```

```
    }
```

```

temp->coef = coefficient;
temp->expon = exponent;
(*ptr)->link = temp;
*ptr = temp;
}
poly_pointer padd(poly_pointer a,poly_pointer b)
{
    poly_pointer front,rear,temp; //temp used to delete first node
    int sum;
    rear = (poly_pointer)malloc(sizeof(poly_node));
    if(!rear)
    {
        fprintf(stderr,"The memory is full");
        exit(1);
    }
    front = rear;
    while(a && b)
    {
        switch(compare(a->expon,b->expon))
        {
            case -1:
                attach(b->coef,b->expon,&rear);
                b = b->link;
                break;
            case 0:
                sum = a->coef + b->coef;
                if(sum)
                    attach(sum,a->expon,&rear);
                a = a->link;
                b = b->link;
                break;
            case 1:

```

```

        attach(a->coef,a->expon,&rear);
        a = a->link;
    }
}
for(;a;a = a->link)
    attach(a->coef,a->expon,&rear);
for(;b;b = b->link)
    attach(b->coef,b->expon,&rear);
rear->link = NULL;    // last node points null
temp = front; front = front->link; free(temp);
return front;
}
int compare(int a,int b)
{
    if(a<b)
        return -1;
    else if(a==b)
        return 0;
    else
        return 1;
}
void createnode(poly_pointer *f)
{
    float c;int e;
    poly_pointer temp,rear; //temp used to create a node
    int count;
    scanf("%d",&count); //to count no of elements in that poly
    rear = *f; //rear points to that poly

    printf("Enter the coefficient and exponent : \n");

    for(int i=0;i<count;i++)

```

```

{
    temp = (poly_pointer)malloc(sizeof(poly_node));
    if(!(temp))
    {
        fprintf(stderr,"Memory Full");
        exit(1);
    }
    scanf("%f %d",&c,&e);
    temp->coef = c;
    temp->expon = e;
    temp->link = NULL;
    if(*f)
        rear->link = temp; //if there is first element
    else
        *f = temp;        //if there is no first element
    rear = temp; //this is to ensure that rear shifts next in for loop
}
}

void display(poly_pointer ptr)
{
    poly_pointer p;
    p = ptr;
    while(p)
    {
        printf("%fX^%d",p->coef,p->expon);
        if(p->link != NULL)
            printf(" + ");
        p = p->link;
    }
}

void erase(poly_pointer *ptr) //this function is to free memory
{

```

```

poly_pointer temp;
while(*ptr)
{
    temp = *ptr;
    *ptr = (*ptr)->link;
    free(temp);
}
printf("\nErased ");
}
int main()
{
    poly_pointer a = NULL,b = NULL,answer;
    printf("Enter number of elements in A : ");
    createnode(&a);
    printf("Enter number of elements in B : ");
    createnode(&b);

    answer = padd(a,b);
    printf("Polynomial after addition is \n");
    display(answer);

    erase(&a);
    erase(&b);
    erase(&answer);
    return 0;
}

```

Output :

```

Enter number of elements in A : 3
Enter the coefficient and exponent :
2 8
4 5
1 3

```

Enter number of elements in B : 4

Enter the coefficient and exponent :

2 9

3 5

2 2

1 0

Polynomial after addition is

$2.000000X^9 + 2.000000X^8 + 7.000000X^5 + 1.000000X^3 + 2.000000X^2 + 1.000000X^0$

Erased

Erased

Erased

11. Write and implement a C program to perform following operations on doubly linked list.

a) Insert a node/create a list (15 marks)

b) Delete a node from a list (15 marks)

c) Display the created list (5 marks)

Program :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef int element; // Define element type for demonstration.
```

```
typedef struct node *node_pointer;
```

```
typedef struct node {
```

```
    node_pointer llink;
```

```
    element item;
```

```
    node_pointer rlink;
```

```
} node;
```

```
node_pointer head = NULL; //global pointer
```

```
void finsert(node_pointer newnode);
```

```
void dinsert(node_pointer newnode,int loc);
```

```
void linsert(node_pointer newnode);
```

```
element fdelete();
```

```
void ddelete(node_pointer delnode);
```

```
element ldelete();
```

```
node_pointer create_node(element item);
```

```
void print_list();
```

```
node_pointer searchval(int v) ;
```

```
void finsert(node_pointer newnode)
```

```
{
```

```

if(!head)
    head = newnode;
else
{
    newnode->rlink = head;
    head = newnode;
    newnode->rlink->llink = newnode;
}
}

void dinsert(node_pointer newnode,int loc)
{
    int i;
    node_pointer ptr;
    if(!head)
    {
        head = newnode;
        return;
    }
    ptr = head;
    for(i=1;i<loc && ((ptr->rlink)!=NULL);i++)
        ptr = ptr->rlink;

    if(ptr == head) //if loc=0
    {
        newnode->rlink = head;
        head->llink = newnode;
        head = newnode;
    }
    else if((ptr->rlink == NULL)&&(i!=loc)) //last element
    {
        ptr->rlink = newnode;
        newnode->llink = ptr;
    }
}

```

```

    }
else
{
    newnode->rlink = ptr;
    newnode->llink = ptr->llink;
    ptr->llink = newnode;
    newnode->llink->rlink = newnode;
}
}

void linsert(node_pointer newnode)
{
    if(!head)
        head = newnode;
    else
    {
        node_pointer ptr = head;
        while(ptr->rlink!=NULL) //stops at last element
            ptr = ptr->rlink;

        ptr->rlink = newnode;
        newnode->llink = ptr;
    }
}

element fdelete()
{
    if(!head)
    {
        printf("LIST EMPTY !!!!!\n"); //if head = null
        return 0;
    }
    node_pointer ptr = head;

```

```

    head = head->rlink;
    head->llink = NULL;
    element val = ptr->item;
    free(ptr);
    return val;
}

void ddelete(node_pointer delnode)
{
    if(delnode == NULL)
        printf("Element Not Found !!!!\n");
    else if((delnode->rlink == NULL)&&(delnode->llink == NULL))
    {
        head = NULL;
        free(delnode);
    }
    else if(delnode->rlink == NULL) //if last element
    {
        delnode->llink->rlink = NULL;
        free(delnode);
    }
    else if(delnode->llink == NULL) //if first element
    {
        head = head->rlink;
        head->llink = NULL;
        free(delnode);
    }
    else
    {
        delnode->llink->rlink = delnode->rlink;
        delnode->rlink->llink = delnode->llink;
        free(delnode);
    }
}

```

```

}
element ldelete()
{
    if(!head)
    {
        printf("LIST EMPTY !!!!!\n");
        return 0;
    }
    node_pointer ptr = head;
    while(ptr->rlink!=NULL) //stop at last element
        ptr = ptr->rlink;

    if(ptr == head) //as deleting from end also deletes memory block pointed by head
        head = NULL;
    else
        ptr->llink->rlink = NULL;
    element val = ptr->item;
    free(ptr);
    return val;
}

node_pointer create_node(element item)
{
    node_pointer newnode = (node_pointer)malloc(sizeof(node));
    newnode->item = item;
    newnode->llink = NULL;
    newnode->rlink = NULL;
    return newnode;
}

void print_list()
{
    if (head == NULL)

```

```

{
    printf("The list is empty.\n");
    return;
}
node_pointer ptr = head;
printf("Doubly Linked List: ");
while (ptr != NULL)
{
    printf("%d ", ptr->item);
    ptr = ptr->rlink;
}
printf("\n");
}
node_pointer searchval(int v) //used to search node
{
    node_pointer ptr = head;
    while(ptr)
    {
        if(ptr->item == v)
            return ptr;
        ptr = ptr->rlink;
    }
    return NULL;
}
int main() {
    int select;int value;int loc;
    element dval;
    node_pointer node;

    printf("Enter \n1 insert at start \n2 insert in middle \n3 insert at end \n4 delete at start \n5
delete in middle \n6 delete at end \n7 display \n8 EXIT\n\n");

    while(1)
    {

```

```
printf("Enter option: ");
scanf("%d",&select);
switch(select)
{
case 1:
    printf("Enter value : ");
    scanf("%d",&value);
    node = create_node(value);
    finser(node);
    break;
case 2:
    printf("Enter value : ");
    scanf("%d",&value);
    printf("Enter location : ");
    scanf("%d",&loc);
    node = create_node(value);
    dinsert(node,loc);
    break;
case 3:
    printf("Enter value : ");
    scanf("%d",&value);
    node = create_node(value);
    linsert(node);
    break;
case 4:
    dval = fdelete();
    printf("%d is Deleted\n",dval);
    break;
case 5:
    printf("Enter the element to be deleted : ");
    scanf("%d",&value);
    node = searchval(value);
```

```

        ddelete(node);
        break;
case 6:
    dval = ldelete();
    printf("%d is Deleted\n",dval);
    break;
case 7:
    print_list();
    break;
case 8:
    printf("Exiting ");
    exit(0);
    break;
default:
    printf("Select the proper value\n");
    }
}
return 0;
}

```

Output :

Enter

1 insert at start

2 insert in middle

3 insert at end

4 delete at start

5 delete in middle

6 delete at end

7 display

8 EXIT

Enter option: 1

Enter value : 10

Enter option: 1

Enter value : 20

Enter option: 7

Doubly Linked List: 20 10

Enter option: 3

Enter value : 30

Enter option: 7

Doubly Linked List: 20 10 30

Enter option: 2

Enter value : 40

Enter location : 2

Enter option: 7

Doubly Linked List: 20 40 10 30

Enter option: 4

20 is Deleted

Enter option: 6

30 is Deleted

Enter option: 7

Doubly Linked List: 40 10

Enter option: 5

Enter the element to be deleted : 45

Element Not Found !!!!!

Enter option: 5

Enter the element to be deleted : 40

Enter option: 7

Doubly Linked List: 10

Enter option: 6

10 is Deleted

Enter option: 7

The list is empty.

Enter option: 8

Exiting

12. Write and implement a C program for the following operations:

a) Construct binary search tree. (20 marks)

b) Binary search tree traversal using: (15 marks)

i) In-order traversal

ii) Pre-order traversal

iii) Post-order traversal

Program :

```
#include<stdio.h>
#include<stdlib.h>
int MAX_SIZE = 25;
typedef struct treenode *tree_pointer;
typedef struct treenode
{
    int item;
    tree_pointer left_child;
    tree_pointer right_child;
}treenode;

tree_pointer modisearch(tree_pointer h,int val) ;
void Tinsert(tree_pointer *head,int val);
void inorder(tree_pointer ptr);
void preorder(tree_pointer ptr);
void postorder(tree_pointer ptr);
tree_pointer search1(tree_pointer root,int key);
tree_pointer search2(tree_pointer root,int key);

//this function search the location to insert given value
tree_pointer modisearch(tree_pointer h,int val)
{
    tree_pointer parent = NULL;
    while(h)
```

```

{
    if(val == h->item)
        return NULL;
    parent = h;
    if(val < h->item)
        h = h->left_child;
    else
        h = h->right_child;
}
return parent;
}

void Tinsert(tree_pointer *head,int val)
{
    tree_pointer temp = modisearch(*head,val);
//temp will point where to insert the given value
    tree_pointer ptr;
    if(temp||!(*head))
    {
        ptr = (tree_pointer)malloc(sizeof(treenode));
        if(!ptr)
        {
            fprintf(stderr,"The memory is full\n");
            exit(1);
        }
        ptr->left_child = NULL;
        ptr->right_child = NULL;
        ptr->item = val;
        if(*head)
        {
            if(val < temp->item)
                temp->left_child = ptr;
            else

```

```

        temp->right_child = ptr;
    }
    else
        *head = ptr;
}

}

void inorder(tree_pointer ptr)
{
    if(ptr)
    {
        inorder(ptr->left_child);
        printf("%d ",ptr->item);
        inorder(ptr->right_child);
    }
}

void preorder(tree_pointer ptr)
{
    if(ptr)
    {
        printf("%d ",ptr->item);
        preorder(ptr->left_child);
        preorder(ptr->right_child);
    }
}

void postorder(tree_pointer ptr)
{
    if(ptr)
    {
        postorder(ptr->left_child);
        postorder(ptr->right_child);
        printf("%d ",ptr->item);
    }
}

```

```

    }
}
tree_pointer search1(tree_pointer root,int key) //recursive
{
    if(!root)
        return NULL;
    if(key == root->item)
        return root;
    if(key < root->item)
        return search1(root->left_child,key);
    return search1(root->right_child,key);
}
tree_pointer search2(tree_pointer root,int key)
{
    while(root)
    {
        if(key == root->item)
            return root;
        if(key < root->item)
            root = root->left_child;
        else
            root = root->right_child;
    }
    return NULL;
}
void main()
{
    int select , value;

    printf("Enter \n1 insertion \n2 search \n3 inorder \n4 preorder \n5 postorder \n6 EXIT\n\n");

    tree_pointer head;

```

```

head = NULL;
while(1)
{
    printf("Enter the choice : ");
    scanf("%d",&select);
    switch(select)
    {
        case 1:
            printf("Enter the value : ");
            scanf("%d",&value);
            Tinsert(&head,value);
            break;
        case 2:
            printf("Enter the value to search : ");
            scanf("%d",&value);

            printf("Recursive : ");
            if(search1(head,value))
                printf("\nElement found \n");
            else
                printf("\nElement not found \n");

            printf("\nIterative : ");
            if(search2(head,value))
                printf("\nElement found \n");
            else
                printf("\nElement not found \n");
            break;
        case 3:
            printf("\nInorder\n");
            inorder(head);
            printf("\n");
    }
}

```

```

        break;
case 4:
    printf("\nPreorder\n");
    preorder(head);
    printf("\n");
    break;
case 5:
    printf("\nPostorder\n");
    postorder(head);
    printf("\n");
    break;
case 6:
    printf("Exiting");
    exit(0);
default:
    printf("Select proper number !\n");
}
}
}

```

Output :

Enter

1 insertion

2 search

3 inorder

4 preorder

5 postorder

6 EXIT

Enter the choice : 1

Enter the value : 50

Enter the choice : 1

Enter the value : 25

Enter the choice : 1

Enter the value : 75

Enter the choice : 1

Enter the value : 10

Enter the choice : 1

Enter the value : 30

Enter the choice : 1

Enter the value : 60

Enter the choice : 1

Enter the value : 90

Enter the choice : 2

Enter the value to search : 60

Recursive :

Element found

Iterative :

Element found

Enter the choice : 2

Enter the value to search : 20

Recursive :

Element not found

Iterative :

Element not found

Enter the choice : 3

Inorder

10 25 30 50 60 75 90

Enter the choice : 4

Preorder

50 25 10 30 75 60 90

Enter the choice : 5

Postorder

10 30 25 60 90 75 50

Enter the choice : 6

Exiting

13. Write a C program to implement the following max heap operations:

a) Insert an item into heap (15 marks)

b) Delete an item from heap (10 marks)

c) Display heap contents (10 marks)

Program :

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#define MAX_ELEMENTS 100
```

```
typedef struct
```

```
{
```

```
    int key;
```

```
}element;
```

```
element heap[MAX_ELEMENTS];
```

```
int n=0;
```

```
int HEAP_FULL(int n);
```

```
int HEAP_EMPTY(int n);
```

```
void insertHeap(element item,int *n);
```

```
element deleteHeap(int *n);
```

```
void disp(int *n);
```

```
void insertHeap(element item,int *n)
```

```
{
```

```
    int i;
```

```
    if(HEAP_FULL(*n))
```

```
    {
```

```
        fprintf(stderr,"THE HEAP IS FULL!!\n");
```

```
        exit(1);
```

```
    }
```

```

i = ++(*n);
while((i!=1) && (item.key > heap[i/2].key))
{
    heap[i] = heap[i/2];
    i/=2;
}
heap[i] = item;
}
element deleteHeap(int *n)
{
    int parent,child;
    element item,temp;
    if(HEAP_EMPTY(*n))
    {
        fprintf(stderr,"THE HEAP IS EMPTY!!\n");
        exit(1);
    }
    item = heap[1];
    temp = heap[(*n)--];
    parent = 1;
    child = 2;

    while(child <= *n)
    {
        if((child < *n) && (heap[child].key)<heap[child+1].key) //compares child in same level
            child++;
        if(temp.key >= heap[child].key) //if last value is greater than heap of child
            break;
        heap[parent] = heap[child];
        parent = child;
        child *= 2;
    }
}

```

```

    heap[parent] = temp;
    return item;
}
void disp(int *n)
{
    printf("Elements are : ");
    if((*n)==0)
        printf("No Elements!");
    for(int i=1;i<=(*n);i++)
        printf("%d ",heap[i].key);
    printf("\n");
}
int HEAP_FULL(int n)
{
    if(n==MAX_ELEMENTS-1)
        return 1;
    return 0;
}
int HEAP_EMPTY(int n)
{
    if(n<=0)
        return 1;
    return 0;
}
void main()
{
    int select;
    element value;
    printf("Enter: \n1 insert \n2 delete \n3 display \n4 EXIT\n\n");
    while(1)
    {
        printf("Enter choice : ");

```

```

scanf("%d",&select);

switch(select)
{
case 1:
    printf("Enter the value : ");
    scanf("%d",&value.key);
    insertHeap(value,&n);
    break;
case 2:
    value = deleteHeap(&n);
    printf("%d is deleted sucessfully\n",value.key);
    break;
case 3:
    disp(&n);
    break;
case 4:
    printf("Exiting...");
    exit(0);
default:
    printf("Select proper number !!\n");
}
}
}

```

Output :

Enter:

1 insert

2 delete

3 display

4 EXIT

Enter choice : 1

Enter the value : 50

Enter choice : 1

Enter the value : 30

Enter choice : 1

Enter the value : 20

Enter choice : 1

Enter the value : 60

Enter choice : 1

Enter the value : 10

Enter choice : 1

Enter the value : 40

Enter choice : 3

Elements are : 60 50 40 30 10 20

Enter choice : 2

60 is deleted sucessfully

Enter choice : 3

Elements are : 50 30 40 20 10

Enter choice : 4

Exiting...

14. Write a C program to implement the following operations over Graph:

a) Construct a Graph (15 marks)

b) Depth First Search (10 marks)

c) Breadth First Search (10 marks)

Program :

```
#include<stdio.h>

#include<stdlib.h>

#define MAX_VERTICES 50

#define FALSE 0

#define TRUE 1

short int visited[MAX_VERTICES];

typedef struct node *node_pointer;

typedef struct node

{

    int vertex;

    node_pointer link;

}node;

node_pointer graph[MAX_VERTICES];

typedef struct queue *queue_pointer;

typedef struct queue

{

    int vertex;

    queue_pointer link;

}queue;

int n=0;

void dfs(int v);

void bfs(int v);

void insertv(node_pointer *g,int i);
```

```

int IS_FULL(queue_pointer temp);
int IS_EMPTY(queue_pointer front);
void addq(queue_pointer *front,queue_pointer *rear,int item);
int deleteq(queue_pointer *front);
void connectedDfs(int nf);
void connectedBfs(int nf);

```

```

void dfs(int v)
{
    node_pointer w;
    visited[v] = TRUE;
    printf("%d ",v);

    for(w=graph[v];w;w=w->link)
        if(!visited[w->vertex])
            dfs(w->vertex);
}

```

```

void bfs(int v)
{
    node_pointer w;
    queue_pointer front,rear;
    front = NULL;
    rear = NULL;
    printf("%d ",v);
    visited[v] = TRUE;
    addq(&front,&rear,v);

    while(front)
    {
        v = deleteq(&front);
        for(w=graph[v];w;w=w->link)
            if(!visited[w->vertex])

```

```

        {
            printf("%d ",w->vertex);
            addq(&front,&rear,w->vertex);
            visited[w->vertex] = TRUE;
        }
    }
}

int IS_FULL(queue_pointer temp)
{
    return !(temp);
}

int IS_EMPTY(queue_pointer front)
{
    return !(front);
}

void addq(queue_pointer *front,queue_pointer *rear,int item)
{
    queue_pointer temp;
    temp = (queue_pointer) malloc(sizeof(queue));
    if(IS_FULL(temp))
    {
        printf("memory full");
        exit(1);
    }
    temp->vertex = item;
    temp->link = NULL;
    if(*front)
        (*rear)->link = temp;
    else
        *front = temp;
    *rear = temp;
}

```

```

int deleteq(queue_pointer *front)
{
    queue_pointer temp = *front;
    int item;
    if(IS_EMPTY(*front))
    {
        printf("Queue is empty");
        exit(1);
    }
    item = temp->vertex;
    *front = temp->link;
    free(temp);
    return item;
}

void insertv(node_pointer *g,int i) //this function is used to create graph
{
    int adj;
    int val;
    printf("Enter number of adjacent node for %d vertice : ",i);
    scanf("%d",&adj);
    printf("Enter values : ");
    for(int j=0;j<adj;j++)
    {
        scanf("%d",&val);
        node_pointer temp;
        temp = (node_pointer)malloc(sizeof(node));
        if(!(temp))
        {
            fprintf(stderr,"Memory full!!");
            exit(1);
        }
    }
}

```

```

temp->vertex = val;
temp->link = NULL;

if(*g)
    (*g)->link = temp;
else
    (*g) = temp;
}
}
void connectedDfs(int nf)
{
    for(int i=0;i<nf;i++)
    {
        if(!visited[i])
        {
            dfs(i);
            printf("\n");
        }
    }
}
void connectedBfs(int nf)
{
    for(int i=0;i<nf;i++)
    {
        if(!visited[i])
        {
            bfs(i);
            printf("\n");
        }
    }
}

```

```

int main()
{
    int nv=0;
    int select;
    printf("Enter the graph details : \n");
    printf("Enter the number of vertices (50 MAX) : ");
    scanf("%d",&nv);
    for(int i=0;i<nv;i++)
    {
        graph[i] = NULL;
        insertv(&graph[i],i);
    }

    printf("Enter : 1 for dfs , 2 for bfs , 3 for EXIT \n\n");
    while(1)
    {
        printf("Enter choice : ");
        scanf("%d",&select);
        switch(select)
        {
            case 1:
                for(int i=0;i<nv;i++)
                    visited[i] = FALSE;
                printf("DFS is : \n");
                connectedDfs(nv);
                break;
            case 2:
                for(int i=0;i<nv;i++)
                    visited[i] = FALSE;
                printf("BFS is : \n");
                connectedBfs(nv);
                break;

```

```

        case 3:
            printf("EXITING.....");
            exit(0);
            break;
        default:
            printf("Select proper number\n");
    }
}
return 0;
}

```

Output :

Enter the graph details :

Enter the number of vertices (50 MAX) : 4

Enter number of adjacent node for 0 vertice : 3

Enter values : 1 2 3

Enter number of adjacent node for 1 vertice : 3

Enter values : 0 2 3

Enter number of adjacent node for 2 vertice : 3

Enter values : 0 1 3

Enter number of adjacent node for 3 vertice : 3

Enter values : 0 1 2

Enter : 1 for dfs , 2 for bfs , 3 for EXIT

Enter choice : 1

DFS is :

0 1 3 2

Enter choice : 2

BFS is :

0 1 3 2

Enter choice : 3

EXITING.....